the data once you know you're the only thread returning the node, *which means the node has already been removed from the queue*. Consequently, passing in the target for the return value by reference is no longer an advantage: you might as well just return by value. If you want to return the value safely, you have to use the other option from chapter 3: return a (smart) pointer to the data value.

If you return a smart pointer, you can just return `nullptr` to indicate that there's no value to return, but this requires that the data be allocated on the heap. If you do the heap allocation as part of the `pop()`, you're *still* no better off, because the heap allocation might throw an exception. Instead, you can allocate the memory when you `push()` the data onto the stack—you have to allocate memory for the `node` anyway. Returning a `std::shared_ptr<>` won't throw an exception, so `pop()` is now safe. Putting all this together gives the following listing.

Listing 7.3. A lock-free stack that leaks nodes